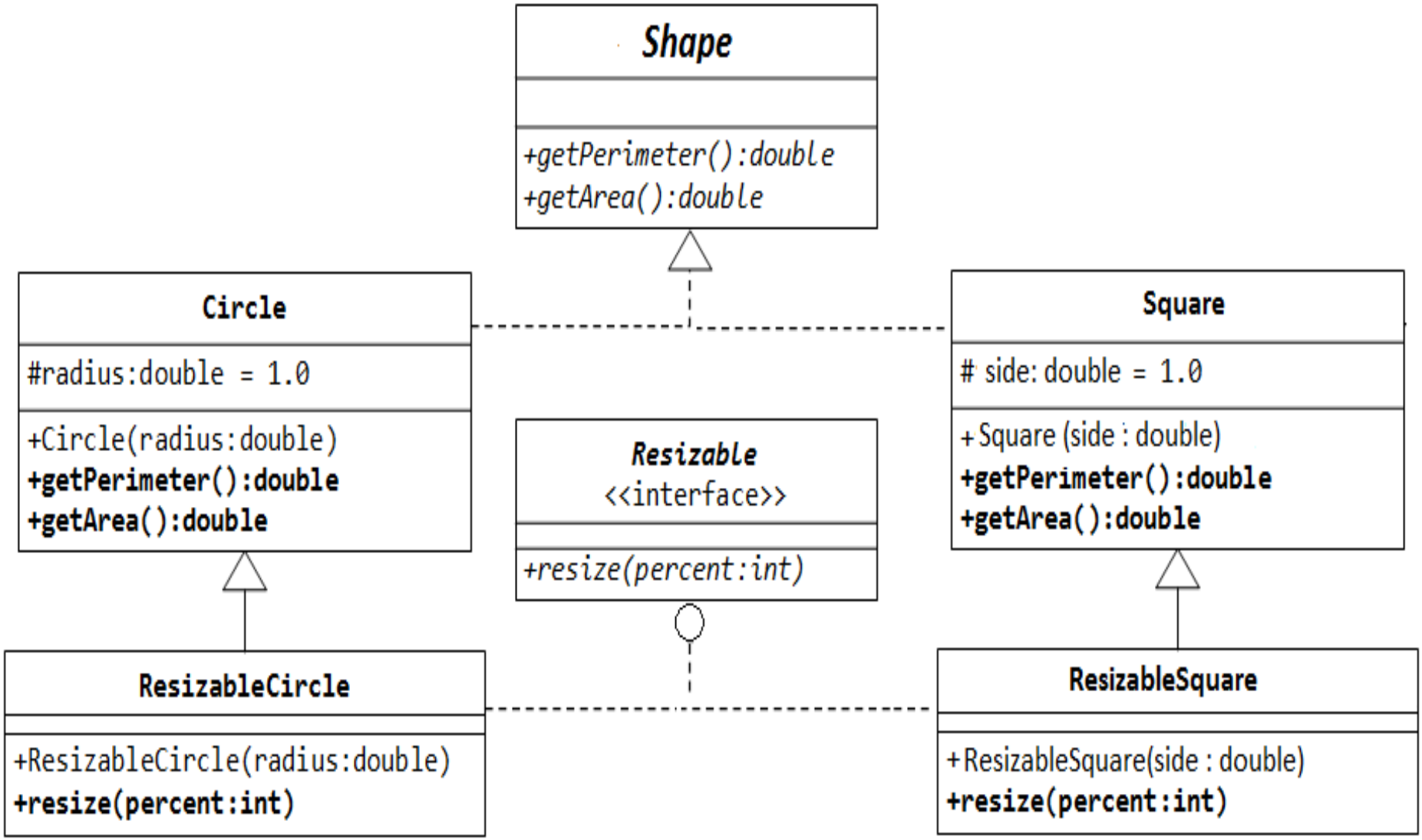
```
class Car{
        private String reg_num; // số đăng ký xe
        private String brand; // hãng
        private int max_pass; // số lượng hành khách tối đa
}
class Truck{
        private String reg_num;
        private String brand;
        private int max_load; // số kg hàng hóa tối đa
}
class Bus{
        private String reg_num;
        private String brand;
        private int max_pass;
        private int number; // mã số tuyến
}
class Motorbike{
        private String reg_num;
        private String brand;
        private int type; // loại xe
}
```

Trong bài toán quản lý thông tin về một số phương tiện giao thông như xe tải, xe buýt, xe con, và xe máy, một giải pháp đơn giản là xây dựng cho mỗi loại xe một class như minh họa.

- Tuy nhiên, giải pháp này không cho phép chương trình ứng dụng thao tác với các loại xe khác nhau một cách đồng bộ, trong khi vẫn có thể phân biệt giữa các loại xe nếu cần.
- Hãy dùng các kĩ thuật kế thừa để sửa giải pháp thiết kế trên sao cho có thể đối xử với tất cả các loại phương tiện giao thông (vehicle) một cách đồng bộ. Nghĩa là tất cả đều nằm trong một cây phân cấp của các lớp đối tượng, trong đó mỗi loại xe vẫn được đại diện bởi một class riêng do chúng có các thuộc tính khác nhau. Cây phân cấp đó cần nhóm xe buýt (bus) và xe con (car) vào một nhóm "xe chở khách" (passenger vehicle).
- Viết các lớp cùng với constructor và các thuộc tính. Viết thêm phương thức input() và print() cho các lớp để nhập và xuất thông tin của đối tượng (sử dụng đa hình).
- Viết hàm main để test lại các lớp trên.


Shape
+getPerimeter():double
+getArea():double
Circle
#radius:double = 1.0
+Circle(radius:double)
+getPerimeter():double
+getArea():double
Square
side: double = 1.0
+ Square (side : double)
+getPerimeter():double
+getArea():double
Resizable
<<interface>>
+resize(percent:int)
ResizableCircle
+ResizableCircle(radius:double)
+resize(percent:int)
ResizableSquare
+ResizableSquare(side : double)
+resize(percent:int)

1/ Cài đặt lớp Circle hoàn chỉnh
Lưu ý: Phải cài đặt trước lớp Shape.

2/ Cài đặt lớp ResizableCircle.
Lưu ý: Phải cài đặt trước giao diện Resizable.
percent là phần trăm, phương thức resize (thay đổi kích cỡ) xuất ra chu vi và diện tích sau khi resize. Nếu percent = 100 thì kích cỡ hình không thay đổi, percent = 50 thì kích cỡ hình nhỏ hơn ban đầu, percent = 150 thì kích cỡ hình lớn hơn ban đầu.

3/ Tương tự câu 1 và câu 2, cài đặt lớp Square và lớp ResizableSquare.

Chợ OOP-Market là một khu kinh doanh sầm uất của thành phố. Mặt bằng chợ được chia thành các sạp có diện tích khác nhau cho tiểu thương thuê. Có ba loại sạp tương ứng với ba loại mặt hàng được phép kinh doanh tại chợ là**: thực phẩm, quần áo, và trang sức**. Đơn giá thuê ở mỗi loại sạp là như nhau: 40.000.000 đồng/m2 /năm. Thông tin chung để quản lý mỗi sạp bao gồm: Số thứ tự sạp, diện tích sạp (m2), doanh thu sạp (đồng)

Vào cuối năm, số tiền mỗi tiểu thương thuê sạp phải đóng = tiền thuê sạp + thuế doanh thu.

Tiền thuê sạp = đơn giá thuê * diện tích sạp.

Thuế doanh thu tùy thuộc vào từng loại sạp và cho bởi bảng sau:

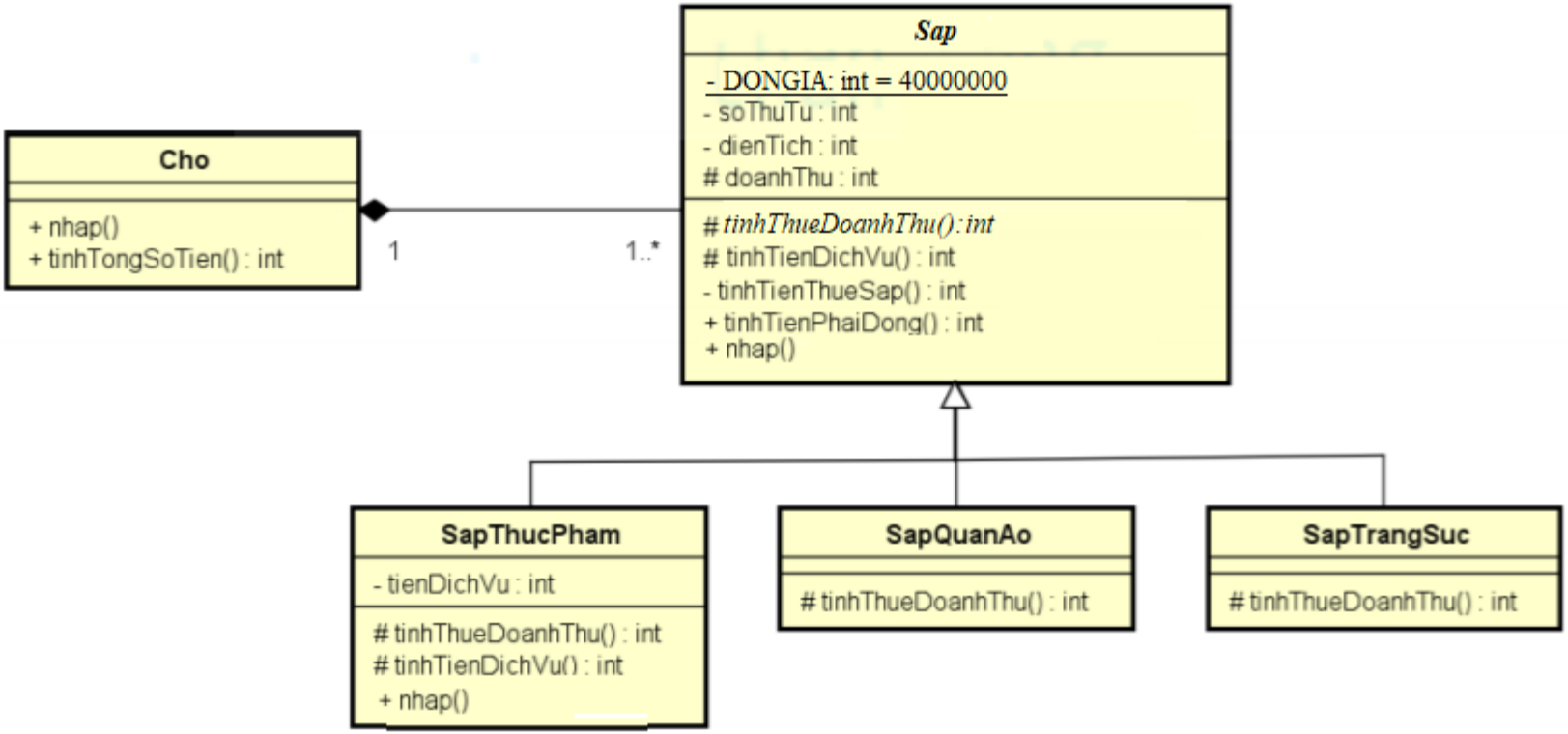
Loại sạp	Thuế doanh thu
Sạp thực phẩm	5%
Sạp quần áo	10%
Sạp trang sức	- Phần doanh thu < 100.000.000 đồng: 20% - Phần doanh thu >= 100.000.000 đồng: 30%

Ngoài ra, những tiểu thương thuê sạp thực phẩm phải đóng thêm tiền sử dụng dịch vụ đông lạnh để bảo quản thực phẩm mà sạp của mình đã sử dụng trong năm (số tiền khác nhau ở từng sạp, tiền dịch vụ là thuộc tính của sạp thực phẩm).

Hãy vẽ sơ đồ lớp và cài đặt chương trình cho phép quản lý số tiền mỗi sạp được thuê trong chợ phải đóng hàng năm như sau:

a) Nhập vào danh sách thông tin các sạp được thuê.

b) Tính tổng số tiền các sạp được thuê phải đóng hàng năm.


Sap
- DONGIA: int = 40000000
- soThuTu : int
- dienTich : int
doanhThu : int
tinhThueDoanhThu(): int
tinhTienDichVu() : int
- tinhTienThueSap() : int
+ tinhTienPhaiDong() : int
+ nhap()
Cho
+ nhap()
+ tinhTongSoTien() : int
1
1..*
SapThucPham
- tienDichVu : int
tinhThueDoanhThu() : int
tinhTienDichVu() : int
+ nhap()
SapQuanAo
tinhThueDoanhThu() : int
SapTrangSuc
tinhThueDoanhThu() : int

Công ty quản lý ca sỹ XYZ cần quản lý các thông tin để tính lương cho các ca sỹ thuộc công ty. Giả sử công ty XYZ chia các ca sỹ thành 2 nhóm: **ca sỹ "chưa" nổi tiếng** và **ca sỹ nổi tiếng**. Thông tin chung của cả 2 nhóm bao gồm:

- Họ tên ca sỹ.
- Số năm làm việc cho công ty.
- Số đĩa đã bán được.

Ngoài ra, ca sỹ nổi tiếng được mời tham gia nhiều Gameshow nên còn có thêm thông tin: số gameshow tham gia. Công ty quy định cách tính và trả lương cho ca sỹ như sau:

- Với **ca sỹ "chưa" nổi tiếng:** Lương = 3.000.000 + 500.000 * số năm làm việc + 1.000 * số đĩa bán được
- Với **ca sỹ nổi tiếng:** Lương = 5.000.000 + 500.000 * số năm làm việc + 1.200 * số đĩa bán được + 500.000 * số Gameshow.

Hãy đề xuất thiết kế hướng đối tượng (vẽ sơ đồ lớp, sử dụng kế thừa và đa hình) và viết chương trình cho phép quản lý danh sách các ca sỹ của Công ty và hỗ trợ tính lương cho ca sỹ theo quy định như trên. Cho phép thực hiện các yêu cầu sau:

a) Nhập danh sách ca sĩ (lưu trữ trong một mảng)

b) Tìm ca sĩ có lương cao nhất trong công ty. Nếu có nhiều ca sĩ có cùng mức lương cao nhất, chỉ cần trả về 1 ca sĩ trong số đó